

Machine Learning 2

Aziz Temirkhanov

Recommender Systems

What is Recommender System?

 **Definition:** Suggest relevant items to users based on past interactions.

 **Applications:** Used in e-commerce, streaming, news, and social platforms.

 **Impact:** Boosts engagement, satisfaction, and business KPIs.

 **ML Problem Formulation:** Modeled as classification, regression, or ranking.

What is Recommender System?

amazon Deliver to Netherlands Computers Search Amazon

All Today's Deals Prime Video Registry Gift Cards Customer Service Sell

EN Hello, sign in Account & Lists Returns & Orders Cart

Results

Check each product page for other buying options.

URBAN ARMOR GEAR
UAG Designed for...
★★★★☆ 548
Shop now

Sponsored

Sponsored

Desk Organizer and Accessories, Computer Monitor Stand Riser with Drawer and 2 Pen Holders, Office Desk Accessories &...
4.6 ★★★★★ (346)
1K+ bought in past month
\$18⁹⁹ Typical: \$19.99
Delivery **Fri, Oct 17**
Ships to Netherlands
Add to cart

Sponsored

7-in-1 Memory Card Reader Universal Card Reader for iPhone/iPad with USB/USB C Connector & SD/TF Card Slots U...
4.2 ★★★★★ (161)
200+ bought in past month
\$14⁹⁹ Typical: \$15.99
Save 10% with coupon
Delivery **Fri, Oct 17**
Ships to Netherlands
Add to cart

Sponsored

Lenovo Legion H410 Wireless Gaming Headset, Bluetooth or 2.4 GHz USB-A Connection, Lightweight (304 Grams),...
3.8 ★★★★★ (10)
100+ bought in past month
\$63⁹⁹ List: \$79.99
Delivery **Fri, Oct 17**
Ships to Netherlands
Add to cart

Sponsored

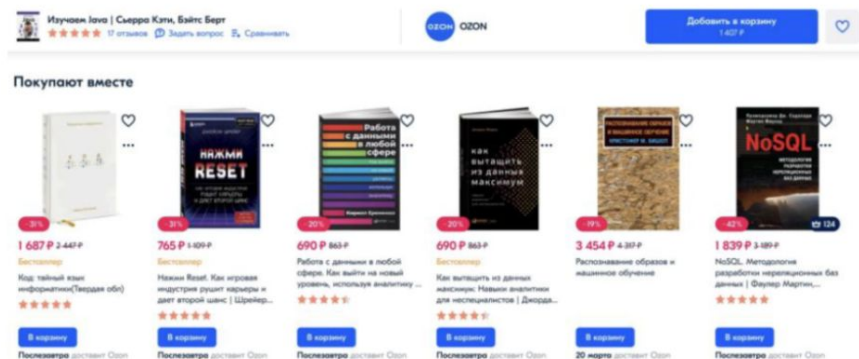
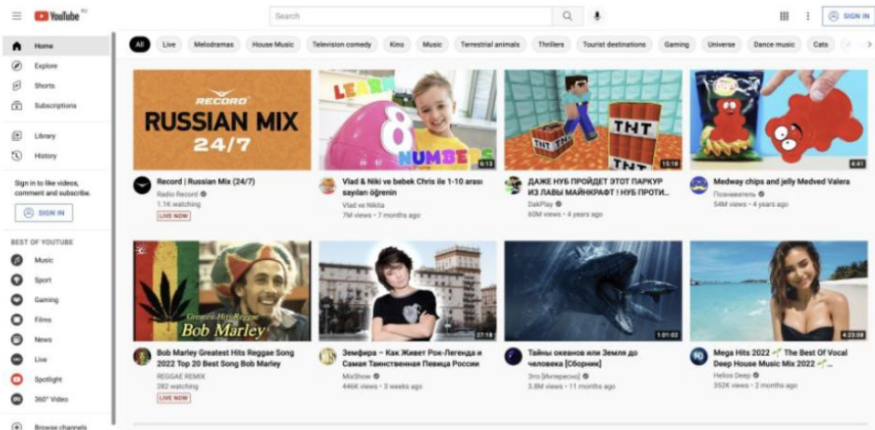
QGeeM USB C Hub, Multiple Converter Adapter 5 in 1, Type C Splitter Extender with Single 8K, Dual 4K/60Hz, 10Gbps Data...
4.2 ★★★★★ (19)
\$49⁹⁹
Delivery **Mon, Oct 20**
Ships to Netherlands
Carbon impact
Add to cart

Sponsored

USB HD Webcam with Built-in Microphone and Rotatable Tripod, 1080P Video, Wide Angle Camera, Privacy Cover, for Desktop PC or...
4.8 ★★★★★ (40)
New on Amazon in past month
\$18⁹⁹ Typical price: \$49.99
Delivery **Fri, Oct 17**
Ships to Netherlands
See options

ONLY Legit! (read instructions) to make sure your data goes into the right place.
Easy Transfer Cable
... Windows

What is Recommender System?



What is Recommender System?

How to tell an item is interesting to a user?

- Item was added to cart
- Listened to the track to the end
- Liked a post or article
- Watched at least half of a video

What is Recommender System?

-  **Interaction Matrix:** Let $R \in \mathbb{R}^{m \times n}$ be the user-item matrix where R_{ui} is the feedback of user u on item i .
-  **Objective:** Learn a function $f(u, i) \rightarrow \hat{r}_{ui}$ that predicts unobserved user-item interactions.
-  **Recommendation Task:** For each user u , recommend top-K items with highest $\hat{r}_{ui}$ not seen before.
-  **Learning Setting:** Can be explicit (ratings) or implicit (clicks, views); supervised or unsupervised.

Explicit Feedback

These are actions that explicitly reflect the user's attitude toward items and content, for example:

- Rating a movie.
- Like or dislike on a track or video.
- Writing a review of a product, article, or film.

Notes:

- This type of data is relatively scarce.
- A dislike doesn't always mean we shouldn't recommend similar items—maybe the category is interesting but that specific item wasn't a fit.

Implicit feedback

Implicit feedback is any other data about user actions on the site. It doesn't express explicit preference, but can serve as a proxy for explicit feedback. For example:

- Purchasing an item
- Viewing an article
- Watch/listen duration

Notes:

- There's much more of this data
- It's less reliable
- Techniques that use it differ from those for explicit feedback

Collaborative Filtering

 **Core Idea:** Users who liked similar items in the past will likely agree in the future.

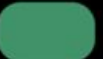
 **No Feature Engineering:** Uses the interaction matrix alone; independent of item or user features.

Sparsity Handling: Relies on neighborhood models or matrix factorization to work with sparse data.

 **Scalability:** Effective for large-scale systems when optimized via ALS or SGD.

Collaborative Filtering

User \ Item										
	1	2	3	4	5	6	7	8	9	10
Петя										
Маша										
Вася										
Катя										

Collaborative Filtering

User-Based Prediction:

$$\hat{r}_{ui} = \bar{r}_u + \frac{\sum_{v \in N(u)} s(u, v)(r_{vi} - \bar{r}_v)}{\sum_{v \in N(u)} |s(u, v)|}$$

Item-Based Prediction:

$$\hat{r}_{ui} = \bar{r}_i + \frac{\sum_{j \in N(i)} s(i, j)(r_{uj} - \bar{r}_j)}{\sum_{j \in N(i)} |s(i, j)|}$$

Latent Factor Model:

$$\hat{r}_{ui} = p_u^T q_i \quad \text{with latent vectors } p_u, q_i$$

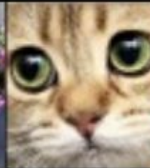
Similarity Function:

$$s(u, v) = \frac{r_u \cdot r_v}{\|r_u\| \|r_v\|}$$

Content-based Filtering

-  **Principle:** Recommend items with similar attributes to those the user liked.
-  **User Profile:** Built from aggregated item features.
-  **Item Similarity:** Measured with cosine or dot product.
-  **Pros & Cons:** Cold-start friendly but over-specialized.

Content-based Filtering

User \ Item										
Петя										
Маша										
Вася										
Катя										

Content-based Filtering

Item Representation:

$$x_i = [x_{i1}, x_{i2}, \dots, x_{id}]$$

User Profile Vector:

$$p_u = \frac{1}{|I_u|} \sum_{i \in I_u} x_i$$

Scoring Function:

$$\hat{r}_{ui} = \cos(p_u, x_i) = \frac{p_u \cdot x_i}{\|p_u\| \|x_i\|}$$

 **Recommendation Rule:** Recommend top-K items with highest predicted similarity scores $\hat{r}_{ui}$.

Matrix Factorization

Matrix Factorization

Users		Items		
		A	B	C
	Ted	4	5	5
	Carol		5	5
	Bob		5	?

		Item			
		A	B	C	D
User	User/ Item				
	Ted		4		3
	Carol	3		2	
	Bob		5		2
	Alice			4	
			?		



T	1.4	.9
C	1.2	1
B	1.5	.9
A	1.2	.8

User



A	B	C	D
1.4	1.3	.9	1.2
.8	11	2	.8

Item

Matrix Factorization

SVD Objective:

$$R_{m \times n} \approx U_{m \times k} \Sigma_{k \times k} V_{k \times n}^T$$

Prediction Rule:

$$\hat{r}_{ui} = \langle p_u, q_i \rangle = p_u^T q_i$$

 **Learning Method:** Optimize latent vectors p_u, q_i by minimizing squared error with regularization.

Regularized Loss:

$$\min_{P, Q} \sum_{(u, i) \in K} (r_{ui} - p_u^T q_i)^2 + \lambda (\|p_u\|^2 + \|q_i\|^2)$$

Matrix Factorization

ALS Idea: Fix Q , solve for P ; fix P , solve for Q alternately.



Objective Function:

$$\min_{P, Q} \sum_{(u, i) \in K} (r_{ui} - p_u^T q_i)^2 + \lambda(\|p_u\|^2 + \|q_i\|^2)$$



Update Rule (User):

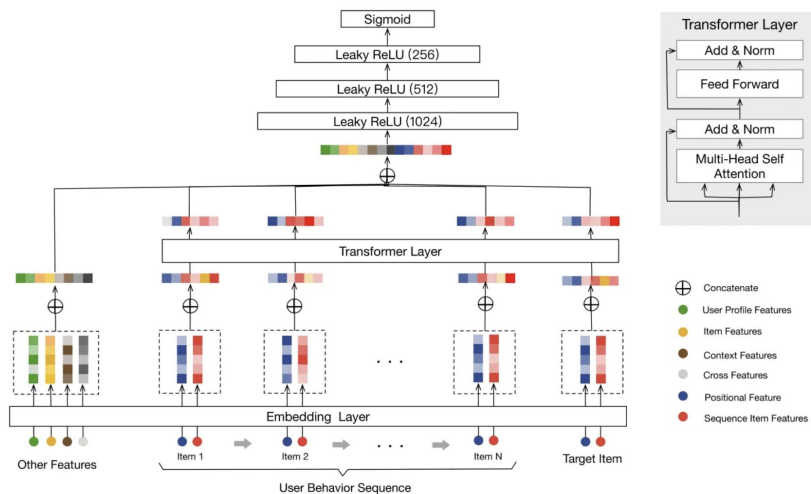
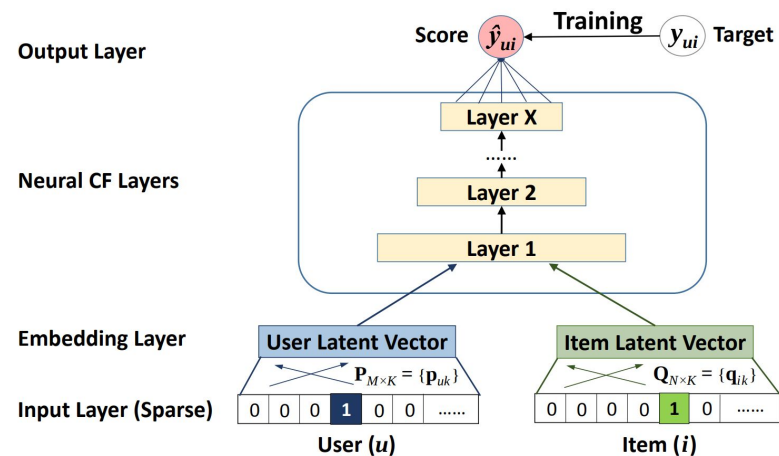
$$p_u = (Q^T Q + \lambda I)^{-1} Q^T r_u$$



Update Rule (Item):

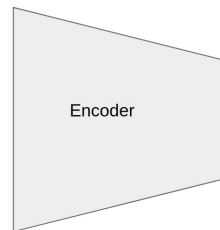
$$q_i = (P^T P + \lambda I)^{-1} P^T r_i$$

And many more...



Vector of users interactions.

1
0
1
1
1
0
.
.
.
0
0



mean vector
2.23
-0.5
1.11
0.76
.
.
.
2.2
0.12
1.01
0.6

Latent representation of user
1.1
-1
1.1
0.5



Predicted scores
1.55
0.67
1.13
2.1
0.3
0.1
.
.
.
1.3
0.7

Metrics

Metrics

🎯 Precision@K:

$$\text{Precision@}K = \frac{|\text{relevant} \cap \text{recommended}_K|}{K}$$

↺ Recall@K:

$$\text{Recall@}K = \frac{|\text{relevant} \cap \text{recommended}_K|}{|\text{relevant}|}$$

📊 MAP (Mean Average Precision):

$$\text{MAP} = \frac{1}{|U|} \sum_{u \in U} \frac{1}{|R_u|} \sum_{k=1}^{|R_u|} P@k(u) \cdot rel_k$$

Metrics

NDCG@K:

$$\text{NDCG@K} = \frac{\text{DCG@K}}{\text{IDCG@K}}, \quad \text{DCG@K} = \sum_{i=1}^K \frac{\text{rel}_i}{\log_2(i+1)}$$

 **Novelty:** Recommends less popular items; often proxied via inverse popularity or exposure frequency.

 **Serendipity:** Measures relevance with unexpectedness; can be operationalized via distance from known items.

Metrics

✂ Definition:

$$\text{Coverage} = \frac{|I_{\text{recommended}}|}{|I|}$$

Fraction of items recommended over all items.

- 🕒 **Temporal Analysis:** Evaluate over intervals (e.g., day, week) to capture dynamics of user/item space.
- ❄ **Cold Start Effect:** New users/items reduce coverage unless handled (e.g., popularity-based bootstrapping).
- 👛 **Impact of Business Constraints:** Restrictions (e.g., filters, quotas) can limit coverage potential.

Metrics

💡 **Goal:** Recommend items that the user is unlikely to already know.

🔍 **Definition via Popularity:**

$$\text{Novelty@K}(u) = \frac{1}{K} \sum_{i \in \text{Top-K}(u)} -\log_2 P(i)$$

where $P(i)$ is the popularity (e.g., frequency) of item i .

👥 **Interpretation:** Higher values indicate less popular (thus more novel) recommendations.

⚖️ **Trade-off:** Excess novelty may hurt relevance; balance with precision is needed.

Metrics

 **Goal:** Recommend varied items, reducing redundancy.

 **Intra-List Diversity (ILD):**

$$\text{ILD}(L) = \frac{1}{|L|(|L| - 1)} \sum_{i \neq j \in L} (1 - \text{sim}(i, j))$$

where $\text{sim}(i, j)$ is cosine or Jaccard similarity.

 **Implication:** Diverse lists expose users to broader catalog regions.

 **Challenge:** Higher diversity can reduce perceived personalization.

Metrics

 **Definition:** Serendipity is the ability to recommend relevant but unexpected items.

 **Formalization:**

$$\text{Serendipity@K}(u) = \frac{1}{K} \sum_{i \in \text{Top-K}(u)} (1 - \text{sim}(i, P_u)) \cdot \text{rel}(u, i)$$

where P_u is a baseline or popular profile.

 **Interpretation:** Combines novelty with relevance.

 **Use Case:** Encourages discoverability and long-tail item exposure.